

Task 3: Firewall Policy Design

Network Security — Practical Task Report

1. Objective

The objective of this task is to design and implement a robust set of firewall rules (inbound and outbound) to protect a web server from unauthorized access, while still allowing the traffic that is necessary for the server to function. In practice, this means the firewall policy has to strike a balance: ports that host legitimate services must remain reachable, while ports associated with insecure, unused, or unnecessary services must be closed off at the network layer.

This task was carried out in a Linux-based environment using iptables as the firewall engine, and the resulting policy was tested and verified using multiple independent tools — Nmap, curl, netcat, and Metasploit — so that no single tool's output is relied upon in isolation.

2. Tools Used

The following tools were used across the scanning, configuration, and verification stages of this task:

- **Kali Linux** — Provided the testing and attack-simulation environment from which scans were launched.
- **Nmap** — Used for TCP port scanning and service detection, both before and after firewall changes.
- **iptables** — The Linux kernel firewall used to define and enforce the INPUT chain rules for this task.
- **curl** — Used to send an actual HTTP request to the web server, verifying application-layer accessibility rather than just port state.
- **netcat (nc)** — Used as an independent method of testing raw TCP connectivity to specific ports.
- **Metasploit (msfconsole)** — Used to run an auxiliary TCP port scanner module as a third, independent verification method.

3. Methodology

The task followed a scan → configure → verify workflow, repeated across two iterations of the firewall policy:

- Step 1 — Baseline scan: An initial Nmap scan established the starting state of ports 22, 80, and 443 before any firewall changes were made.
- Step 2 — Service check: The web server (Nginx) status was confirmed to ensure port 80 corresponded to a genuinely running service.
- Step 3 — Iteration 1 (permissive baseline): iptables REJECT rules were added for ports 23 (Telnet) and 443 (HTTPS), while leaving the default INPUT policy at ACCEPT.
- Step 4 — Independent verification: The new rules were verified using four separate methods — an Nmap re-scan, a curl request to the web server, netcat connection tests, and a Metasploit port scan — to avoid relying on a single tool's result.
- Step 5 — Iteration 2 (final hardened policy): The policy was strengthened by explicitly allowing only ports 22 and 80 via ACCEPT rules, then changing the default INPUT policy from ACCEPT to DROP, so that any traffic not explicitly permitted is blocked by default.
-

4. Port Status Summary

The table below summarizes the observed state of each relevant port before firewall rules were applied (Screenshot 1) and after Iteration 1 was applied and re-scanned (Screenshot 5).

Port	Service	State — Before Rules (Scr. 1)	State — After Iteration 1 (Scr. 5)
22/tcp	SSH	Open	Open
23/tcp	Telnet	Not scanned	Closed
80/tcp	HTTP	Open	Open
443/tcp	HTTPS	Closed	Closed

Note: Port 23 was not included in the very first scan (Screenshot 1), which only covered ports 22, 80, and 443; it was added in the follow-up scan shown in Screenshot 5, after the REJECT rule for port 23 had already been applied.

5. Firewall Rule Sets — Iteration Comparison

Two distinct iptables configurations appear across the evidence collected for this task. The table below lists the rules exactly as they appear in the corresponding screenshots.

5.1 Iteration 1 — Permissive Strategy (Screenshots 3 & 4)

Chain / Rule	Value
Default INPUT policy	ACCEPT
Default FORWARD policy	ACCEPT
Default OUTPUT policy	ACCEPT
Rule 1	-A INPUT -p tcp --dport 23 -j REJECT --reject-with icmp-port-unreachable
Rule 2	-A INPUT -p tcp --dport 443 -j REJECT --reject-with icmp-port-unreachable (confirmed in Screenshot 9)

5.2 Iteration 2 — Final Hardened Strategy (Screenshot 9)

Chain / Rule	Value
Default INPUT policy	DROP
Default FORWARD policy	ACCEPT
Default OUTPUT policy	ACCEPT
Rule 1	-A INPUT -p tcp --dport 22 -j ACCEPT
Rule 2	-A INPUT -p tcp --dport 80 -j ACCEPT
Rule 3	-A INPUT -p tcp --dport 23 -j REJECT --reject-with icmp-port-unreachable
Rule 4	-A INPUT -p tcp --dport 443 -j REJECT --reject-with icmp-port-unreachable

The two iterations reflect different firewall design philosophies. Iteration 1 is permissive by default (everything allowed unless explicitly rejected), whereas Iteration 2 is deny-by-default (everything blocked unless explicitly allowed). Iteration 2 — the deny-by-default configuration — is treated as the final policy for this task, as it follows the principle of least privilege more closely.

6. Cross-Tool Verification Summary

To avoid relying on a single tool, the firewall behaviour was independently confirmed using four different methods. The table below consolidates the result each tool reported for each port.

Port	Nmap (Scr. 5)	curl (Scr. 6)	netcat (Scr. 7)	Metasploit (Scr. 8)
22/tcp	Open	N/A	N/A	TCP OPEN
23/tcp	Closed	N/A	Connection refused	Not explicitly shown
80/tcp	Open	HTTP 200 OK	N/A	TCP OPEN
443/tcp	Closed	N/A	Connection refused	Not explicitly shown

"N/A" indicates the tool was not used to test that specific port in the corresponding screenshot (for example, curl was only used against port 80, and netcat was only run against ports 23 and 443). "Not explicitly shown" indicates the Metasploit scan output for that port is not visible in Screenshot 8, as explained in Section 8.

7. Evidence and Detailed Observations

This section presents the screenshots collected during the task in chronological order, each followed by a detailed explanation of what the evidence shows and why that step was necessary. Each subsection also notes the exact command executed and the timestamp recorded in the terminal, so the sequence of actions can be reconstructed independently of this report.

Where relevant, the explanation also connects the observation back to the port status and rule tables in Sections 4 and 5, so that the raw terminal evidence and the summarized tables can be cross-checked against each other.

7.1 Baseline Port Scan (Before Firewall Rules)

```
ubuntu@sandbox:~$ nmap -p 22,80,443 localhost 66 hostname 66 date
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-05 08:59 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00002s latency).
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   closed https
Nmap done: 1 IP address (1 host up) scanned in 0.03 seconds
d062676716ff
Sun Jul 5 08:59:08 UTC 2026
```

Screenshot 1 — Nmap scan of ports 22, 80, 443 on localhost, prior to firewall rule changes (06:59:08 UTC).

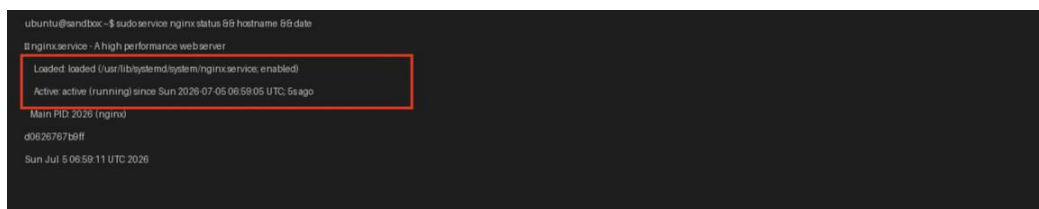
Before any firewall rule was introduced, a baseline Nmap scan was run against the local host to establish the starting state of the three ports relevant to this task: 22 (SSH), 80 (HTTP), and 443 (HTTPS).

The scan reports port 22 as open, running SSH — this is the administrative access channel and is expected to remain reachable throughout the exercise.

Port 80 is reported open, running HTTP — this corresponds to the Nginx web server that the firewall policy is ultimately designed to protect.

Port 443 is reported closed at this stage, before any firewall rule targeting it had been applied. This establishes that HTTPS was never actually configured or listening on this host, which is an important baseline fact used later when interpreting the firewall rule for port 443.

7.2 Web Server Status Verification

A terminal window showing the status of the Nginx service. The output is as follows:

```
ubuntu@handbox:~$ sudo service nginx status 80 hostname: 80 date
nginx.service - A high performance web server
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled)
Active: active (running) since Sun 2026-07-05 06:59:05 UTC; 5s ago
Main PID: 2026 (nginx)
d0626767b6ff
Sun Jul 5 06:59:11 UTC 2026
```

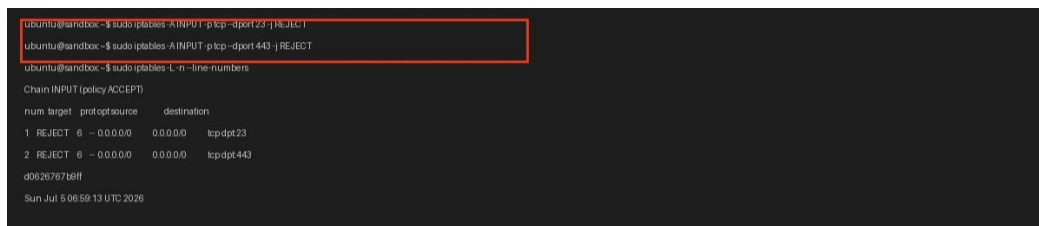
A red rectangle highlights the 'Loaded' and 'Active' status lines.

Screenshot 2 — Nginx service status confirming the web server is active and running (06:59:11 UTC).

To confirm that the service the firewall is meant to protect is genuinely running, the Nginx service status was checked using the system service manager.

The output confirms the nginx.service unit is loaded, enabled, and active (running), with an associated process ID. This establishes that port 80 traffic observed in the Nmap scan corresponds to a real, running web server rather than an idle or misconfigured listener.

7.3 Firewall Rule Implementation — Iteration 1 (Permissive Baseline)

A terminal window showing the configuration of iptables rules. The output is as follows:

```
ubuntu@handbox:~$ sudo iptables -A INPUT -p tcp --port 23 -j REJECT
ubuntu@handbox:~$ sudo iptables -A INPUT -p tcp --port 443 -j REJECT
ubuntu@handbox:~$ sudo iptables -L -n --line-numbers
Chain INPUT (policy ACCEPT)
num target prot opt source destination
1 REJECT 6 -- 0.0.0.0 0.0.0.0 tcp dpt:23
2 REJECT 6 -- 0.0.0.0 0.0.0.0 tcp dpt:443
d0626767b6ff
Sun Jul 5 06:59:13 UTC 2026
```

A red rectangle highlights the two iptables commands used to add the REJECT rules.

Screenshot 3 — Adding iptables REJECT rules for ports 23 (Telnet) and 443, and listing rules with line numbers (06:59:13 UTC).

The first iteration of the firewall policy takes a permissive-by-default approach: the default INPUT chain policy is left at ACCEPT, and two explicit REJECT rules are added instead — one for port 23 (Telnet), and one for port 443 (HTTPS).

Blocking Telnet (port 23) is a standard hardening step, since Telnet transmits credentials and data in plaintext and is considered insecure for any production system.

Blocking port 443 at the firewall level, in combination with the baseline finding from Screenshot 1 that no service was listening on 443, reinforces that HTTPS is not intended to be reachable on this host at this stage.

The iptables -L -n --line-numbers output confirms both rules are registered in the INPUT chain, each targeting its respective TCP destination port with a REJECT action.

```
ubuntu@sandbox:~$ sudo iptables -S 88 hostname 88 date
-P INPUT ACCEPT
-P FORWARD ACCEPT
-P OUTPUT ACCEPT
-A INPUT -p tcp -m tcp --dport 23 -j REJECT --reject-with icmp-port-unreachable
-A INPUT -p tcp -m tcp --dport 443 -j REJECT --reject-with icmp-port-unreachable
d0626767b6ff
Sun Jul 5 06:59:16 UTC 2026
```

Screenshot 4 — *iptables -S* output after adding the *REJECT* rules (06:59:16 UTC).

The *iptables -S* output restates the active rule set in command-line format. The default policies for INPUT, FORWARD, and OUTPUT are all ACCEPT at this stage, consistent with the permissive-baseline strategy, and the REJECT rule for port 23 is clearly visible with the *icmp-port-unreachable* response type.

The corresponding REJECT rule for port 443 is present in the same output but is partially obscured by the terminal cursor in this particular capture (see Section 6, Notes and Limitations). Its presence is independently confirmed later in the final configuration shown in Screenshot 9.

7.4 Verification — Nmap Re-scan

```
ubuntu@sandbox:~$ nmap -p 22,23,80,443 localhost 88 hostname 88 date
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-05 06:59 UTC

PORT      STATE SERVICE
22/tcp    open  ssh
23/tcp    closed telnet
80/tcp    open  http
443/tcp   closed https
Nmap done: 1 IP address (1 host up) scanned in 0.03 seconds
d0626767b6ff
Sun Jul 5 06:59:18 UTC 2026
```

Screenshot 5 — *Nmap* re-scan of ports 22, 23, 80, 443 confirming port 23 is now closed (06:59:18 UTC).

After applying the Iteration 1 rules, the port scan was repeated, this time including port 23 to directly test the new rule.

The results show port 22 still open (SSH unaffected), port 23 now closed (Telnet successfully blocked), port 80 still open (web server still reachable), and port 443 closed. This is the first independent confirmation that the firewall change achieved its intended effect without disrupting the services that should remain available.

7.5 Verification — Allowed Service Accessibility

```
ubuntu@sandbox:~$ curl -I http://localhost 88 hostname 88 date
HTTP/1.1 200 OK
Server: nginx/3.24.0.Ubuntu
Date: Sun, 05 Jul 2026 06:59:21 GMT
d0626767b6ff
Sun Jul 5 06:59:21 UTC 2026
```

Screenshot 6 — *curl -I http://localhost* showing *HTTP/1.1 200 OK*, confirming port 80 remains accessible (06:59:21 UTC).

Beyond confirming that port 80 shows as "open" in a port scan, an actual HTTP request was made using *curl* to verify the web server responds correctly at the application layer.

The response *HTTP/1.1 200 OK*, together with the *Server: nginx* header, confirms that legitimate web traffic is not only allowed through the firewall but is being served correctly end-to-end — satisfying the task requirement of allowing necessary traffic while restricting unauthorized access.

7.6 Verification — Netcat Cross-check

```
ubuntu@airdbox:~$ nc -zv localhost 23; nc -zv localhost 443; hostname; date
nc connect to localhost (127.0.0.1) port 23 (tcp) failed: Connection refused
nc connect to localhost (127.0.0.1) port 443 (tcp) failed: Connection refused
d0626767b9ff
Sun Jul 5 06:59:27 UTC 2026
```

Screenshot 7 — Netcat (nc) checks on ports 23 and 443 both returning "Connection refused" (06:59:27 UTC).

As a second, independent verification method distinct from Nmap, netcat (nc -zv) was used to attempt raw TCP connections to ports 23 and 443.

Both attempts returned "Connection refused", corroborating the Nmap findings using a different tool and confirming that the blocking behaviour is not specific to how Nmap interprets closed ports.

7.7 Verification — Metasploit Port Scan

```
ubuntu@d0626767b9ff:~$ msfconsole -q -x "use auxiliary/scanner/portscan/tcp; set RHOSTS 127.0.0.1; set PORTS
[*] 127.0.0.1:22 - TCP OPEN
[*] 127.0.0.1:80 - TCP OPEN
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
ubuntu@d0626767b9ff:~$ date
Sun Jul 5 06:59:32 UTC 2026
ubuntu@d0626767b9ff:~$
```

Screenshot 8 — Metasploit auxiliary/scanner/portscan/tcp module results for RHOSTS 127.0.0.1 (06:59:32 UTC).

As a third verification method, and one commonly used in offensive-security style assessments, the Metasploit auxiliary/scanner/portscan/tcp module was run against 127.0.0.1.

The scan reports ports 22 and 80 as TCP OPEN. The exact value supplied to the module's PORTS variable is not fully visible in this capture, so while 22 and 80 are explicitly confirmed open, no explicit CLOSED status for ports 23 or 443 is shown in this particular result (see Section 6).

7.8 Firewall Rule Implementation — Final (Hardened) Configuration

```
ubuntu@d0626767b9ff:~$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
ubuntu@d0626767b9ff:~$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
ubuntu@d0626767b9ff:~$ sudo iptables -P INPUT DROP
ubuntu@d0626767b9ff:~$ sudo iptables -S
-P INPUT DROP
-P FORWARD ACCEPT
-P OUTPUT ACCEPT
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 80 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 23 -j REJECT --reject-with icmp-port-unreachable
-A INPUT -p tcp -m tcp --dport 443 -j REJECT --reject-with icmp-port-unreachable
ubuntu@d0626767b9ff:~$ date
Sun Jul 5 06:59:36 UTC 2026
ubuntu@d0626767b9ff:~$
```

Screenshot 9 — Final iptables configuration: explicit ACCEPT rules for ports 22 and 80, default INPUT policy set to DROP, and REJECT rules for ports 23 and 443 (06:59:36 UTC).

The final stage of the task moves from a permissive-by-default policy to a deny-by-default policy, which is generally regarded as a stronger security posture for a system exposed to untrusted networks.

Two explicit ACCEPT rules are added for ports 22 and 80 — the only two services intended to be reachable. The default INPUT chain policy is then changed from ACCEPT to DROP, meaning any traffic not explicitly matched by an earlier rule is silently dropped rather than allowed through.

The final iptables -S output confirms this configuration: -P INPUT DROP as the default policy, followed by explicit ACCEPT rules for ports 22 and 80, and the REJECT rules for ports 23 and 443 carried over from Iteration 1. This is the configuration considered final for this task.

8. Notes and Limitations

- Port 443 was already closed at the service level before any firewall rule was applied (Screenshot 1). The subsequent iptables REJECT rule for port 443 is therefore a defense-in-depth measure rather than the sole reason the port is inaccessible — no HTTPS service was ever listening on this host.
- In Screenshot 4, the REJECT rule for port 443 is partially obscured by the terminal cursor. This rule's existence is independently confirmed in the final configuration shown in Screenshot 9, where both the port 23 and port 443 REJECT rules are fully visible.
- Two firewall strategies were tested during this task: an initial permissive approach (default INPUT policy ACCEPT, with explicit REJECT rules for ports 23 and 443) and a final hardened approach (default INPUT policy DROP, with explicit ACCEPT rules for ports 22 and 80). These are documented separately in Section 5 rather than merged, to avoid presenting them as a single, unchanging rule set.
- In Screenshot 8, the exact value passed to the Metasploit PORTS variable is not fully visible due to how the terminal window was captured. The scan results explicitly list ports 22 and 80 as TCP OPEN; no explicit CLOSED status for ports 23 or 443 appears in this particular capture, so their status is inferred only from their absence in the "OPEN" results, not from a direct statement in the tool's output.

9. Outcome and Conclusion

By the end of this task, the web server (Nginx, serving HTTP on port 80) remained fully accessible, and the administrative SSH service on port 22 was preserved. Meanwhile, unauthorized access via Telnet (port 23) and the unused HTTPS port (443) was blocked at the firewall layer, and this was confirmed independently across four different tools rather than relying on a single scan.

The final configuration adopted a deny-by-default (DROP) policy with explicit ACCEPT rules for only the two required ports. This is a stronger and more maintainable security posture than a permissive-by-default policy, since any future service that is accidentally left running will not be reachable unless a rule is deliberately added for it. This outcome satisfies the task's objective of protecting the web server from unauthorized access while still allowing the traffic necessary for it to function, and reflects the intended learning outcome of acquiring practical, industry-relevant firewall configuration skills.